Software Requirements Specification (SRS)

VaultKeep – Digital Warranty & Service Book

Document Version: 1.0

Date: July 23, 2026

Department of Computer Engineering

Zeal College of Engineering and Research

1.0 Introduction

1.1 Purpose

The purpose of this document is to define the functional and non-functional requirements for the **VaultKeep – Digital Warranty & Service Book** application. This SRS will serve as a foundational guide for developers, designers, and testers to ensure the system is built according to specified business needs and constraints.

1.2 Project Vision & Scope

VaultKeep is a web-based platform designed to securely centralize the storage of digital warranty cards, purchase invoices, and service histories for electronic products. The scope of this project includes developing a responsive web application where users can upload documents, track warranty expirations, receive timely email reminders, and generate QR codes for quick product retrieval. The system is entirely contained within a local deployment environment using XAMPP and does not rely on third-party cloud integrations or artificial intelligence for its core features.

1.3 Technology Stack

- **Frontend:** HTML5, CSS3, JavaScript (ES6), Bootstrap 5, AJAX
- **Backend:** PHP 8.x
- **Database:** MySQL
- **Server:** XAMPP (Apache)
- **Libraries:** PHPMailer, TCPDF, Chart.js, SweetAlert2, PHP QR Code Generator

2.0 Problem Statement & Proposed Solution

2.1 The Problem

Consumers frequently face challenges in managing post-purchase documentation for electronic devices. Common issues include:

- **Lost Invoices & Warranty Cards:** Physical paper degrades, gets misplaced, or accidentally discarded.
- **Missed Deadlines:** Users forget warranty expiration dates, losing out on free repairs or replacements.
- **Scattered Service Records:** Difficulty tracking when a device was last serviced and what was repaired.

2.2 Proposed Solution (VaultKeep)

VaultKeep solves this by digitizing the entire post-purchase lifecycle. Users create an account, register their products, and upload digital copies (PDF/Images) of their bills and warranty cards. The system actively monitors warranty end dates and automatically sends email alerts before the warranty expires, ensuring users maximize their consumer rights without dealing with physical paper clutter.

3.0 Functional Requirements

3.1 User Module

Feature	Description
Authentication	Secure Registration, Login, and Forgot Password functionality.
Dashboard	A central hub displaying total products, active warranties, expired warranties, and upcoming service dates.
Add Product	Form to enter product details (Name, Brand, Serial Number, Purchase Date, Warranty Period).
Document Upload	Secure upload of Invoices and Warranty Cards (PDF, JPG, PNG).
QR Generation	Automatically generates a downloadable QR code for each product. Scanning the code links back to the product's digital vault entry.
Service History	Users can log maintenance and repair records for individual products.
Reminders	System evaluates dates and triggers PHPMailer to send alerts 30, 15, and 3 days before warranty expiry.
Export/Reports	Users can download a summary of their products and warranties as a PDF using TCPDF.

3.2 Admin Module

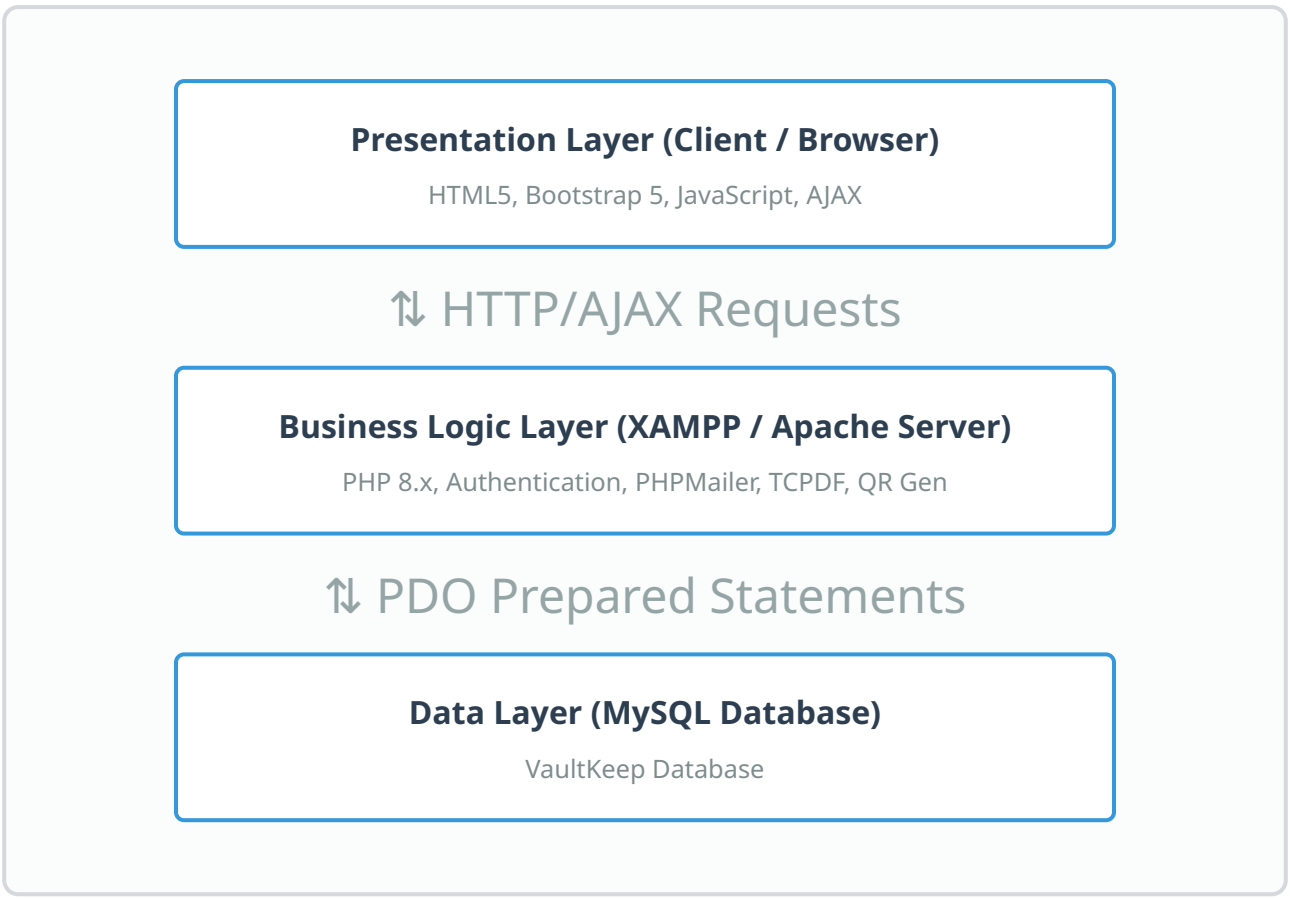
Feature	Description
Admin Dashboard	High-level metrics showing total registered users, total products stored, and system usage statistics (visualized via Chart.js).
User Management	Ability to view, suspend, or delete user accounts if they violate terms of service.
Category Management	Admin can add or remove product categories (e.g., Laptops, Home Appliances, Mobile Phones) available in the dropdown menus.
System Settings	Configuration of global settings, such as reminder thresholds.

4.0 Non-Functional Requirements

- **Performance:** The system must load dashboard data within 2 seconds. Document uploads should process within 5 seconds for files up to 5MB.
- **Security:** Passwords must be hashed using `password_hash()` in PHP. All database queries must use Prepared Statements (PDO) to prevent SQL Injection. Session hijacking prevention must be implemented.
- **Usability:** The UI must be fully responsive, scaling flawlessly from desktop monitors to mobile screens using Bootstrap 5 grid systems.
- **Availability:** Designed to run continuously on the local XAMPP server with zero planned downtime during operational hours.
- **Capacity:** The MySQL database should comfortably handle at least 10,000 product records and associated metadata without performance degradation.

5.0 System Architecture

The application follows a standard **Three-Tier Architecture**, separating the presentation layer, the business logic, and the database.



6.0 Use Case Specifications

6.1 Use Case: Add Product & Upload Bill

- **Actors:** User
- **Preconditions:** User must be logged into the system.
- **Main Flow:**
  1. User navigates to "Add Product" page.
  2. User enters product details.
  3. User selects the bill/warranty file from their device.
  4. System validates file size and extension.
  5. System saves the file to the server and records the path in the database.
- **Exception Flow:** If the file size exceeds 5MB or is an invalid format (e.g., .exe), the system rejects the upload and shows an error prompt.
- **Post Conditions:** The product appears on the user's dashboard and the warranty countdown begins.

7.0 Database Design

7.1 Entity Relationship (ER) Concept

- **Users** (1) can have (M) **Products**.
- **Products** (1) can belong to (1) **Category**.
- **Products** (1) can have (M) **Service Histories**.

7.2 Data Tables Dictionary

Table: `users`

Column Name	Data Type	Key/Constraint	Description
user_id	INT	Primary Key, Auto Inc	Unique ID for the user
full_name	VARCHAR(100)	Not Null	User's full name
email	VARCHAR(150)	Unique, Not Null	Email address for login and alerts
password	VARCHAR(255)	Not Null	Hashed password
created_at	TIMESTAMP	Default CURRENT_TIMESTAMP	Account creation date

Table: `products`

Column Name	Data Type	Key/Constraint	Description
product_id	INT	Primary Key, Auto Inc	Unique product ID
user_id	INT	Foreign Key (users)	Owner of the product
product_name	VARCHAR(150)	Not Null	Name/Model of the product
purchase_date	DATE	Not Null	Date of purchase
expiry_date	DATE	Not Null	Calculated warranty expiry
bill_path	VARCHAR(255)	Nullable	File path to uploaded invoice
qr_code_path	VARCHAR(255)	Nullable	File path to generated QR image

8.0 Testing Strategy

Testing Phase	Description
Unit Testing	Testing individual PHP functions (e.g., checking if the date calculation logic correctly identifies warranties expiring in 30 days).
Integration Testing	Ensuring that third-party libraries (PHPMailer, TCPDF) communicate correctly with the core PHP backend.
Security Testing	Attempting SQL injection on login/upload forms and verifying that file upload validators properly block malicious scripts.
User Acceptance Testing	Having end-users navigate the UI to confirm that adding a product, uploading a bill, and downloading a report is intuitive and smooth.

9.0 Project Timeline (Gantt-Style Schedule)

Phase	Key Activities
Phase 1: Planning	Requirements gathering, SRS documentation, UI wireframing.
Phase 2: Database & UI	Creating MySQL schema, writing HTML/CSS/Bootstrap frontend.
Phase 3: Backend Logic	PHP CRUD operations, User Authentication, Session management.
Phase 4: Integrations	Implementing PHPMailer, TCPDF, QR Code Generator.
Phase 5: Testing	Bug fixing, security audits, responsiveness checks.
Phase 6: Deployment	Final XAMPP deployment, project report writing, final presentation prep.

10.0 Future Scope

While the current version of VaultKeep is robust and self-contained, future iterations may include:

1. **Optical Character Recognition (OCR):** Automatically extracting purchase dates, amounts, and serial numbers directly from scanned bill images.
2. **Cloud Storage Integration:** Allowing users to back up their documents directly to Google Drive or AWS S3.
3. **Dedicated Mobile Application:** Developing a native Android/iOS app with barcode and physical QR scanning capabilities using the device camera.
4. **Google Calendar API Integration:** Syncing warranty expiry dates directly to the user's personal calendar.

11.0 Conclusion

The **VaultKeep** application provides significant business and personal value by solving a highly relatable problem: the mismanagement of post-purchase product documentation. By

utilizing a lightweight, accessible, and secure local tech stack (PHP/MySQL/Bootstrap), the system guarantees that users will never again miss a warranty claim due to lost paperwork or forgotten dates. Its technical advantages—such as automated email reminders and instant QR code retrieval—make it a highly efficient, modern digital vault ready for successful academic presentation and potential real-world deployment.